Licenciatura en Sistemas de Información
SO - Sistemas Distribuidos
Trabajo Práctico: Sincronización en Sistemas Distribuidos
Alumno/Grupo: gonzalo romero, valentin romero, luciano sangalli y zaragoza nahuel
Link al repositorio: https://github.com/NahuelZarago/SistemasDistribuidos

1. Respuestas Teóricas (Parte I)

1. ¿Cuál es la diferencia entre un reloj físico y un reloj lógico?
Un reloj físico mide el tiempo real (segundos, milisegundos) buscando precisión cronológica. Un reloj lógico es un contador de software que no mide tiempo real, sino el orden secuencial de los eventos (qué ocurrió antes que qué).

2. ¿Qué es la deriva de un reloj y cómo afecta a los sistemas distribuidos?
Es la variación en la frecuencia con la que un reloj de hardware cuenta el tiempo respecto a un reloj de referencia perfecto. En sistemas distribuidos, causa que los nodos se desincronicen progresivamente, dificultando el registro coherente de eventos conjuntos.

3. ¿Qué es el Tiempo Atómico Internacional (TAI) y cuál es su relación con UTC?
El TAI mide el tiempo exacto mediante transiciones atómicas sin ajustes. El UTC toma el TAI como base y le aplica "segundos intercalares" periódicos para mantenerlo sincronizado con la rotación de la Tierra.

4. ¿Por qué resulta difícil establecer un orden global de eventos utilizando únicamente relojes físicos?
Porque no existe un reloj global perfecto y las redes introducen latencia impredecible. Siempre habrá un margen de error entre distintos servidores; si dos eventos ocurren casi al mismo tiempo, el error físico impide asegurar matemáticamente cuál ocurrió primero.

5. ¿Cómo funciona el algoritmo de Cristian? Explique sus ventajas y limitaciones.
Un cliente consulta la hora a un servidor. Al recibirla, asume que el tiempo de ida fue igual al de vuelta, ajustando su reloj a `Hora_Servidor + (RTT / 2)`. Es simple, pero falla si el servidor cae y pierde precisión si la latencia de la red es asimétrica.

6. ¿Cómo funciona el algoritmo de Berkeley? ¿Qué función cumple el nodo coordinador?
Un nodo (coordinador) solicita la hora a todos los demás, calcula un promedio (descartando relojes excesivamente desfasados) y envía a cada nodo la corrección que debe aplicar. El coordinador lidera el consenso interno, sin depender de un servidor externo de tiempo real.

1. Respuestas Teóricas (Parte II)

7. ¿Qué diferencias existen entre los algoritmos de Cristian y Berkeley?
Cristian es pasivo (clientes sincronizan contra una fuente de tiempo "real" externa). Berkeley es activo (busca consenso interno promediando los relojes de todos para que tengan la misma hora local, aunque difiera del tiempo mundial).

8. ¿Qué es la relación happens-before definida por Lamport?
Es una relación matemática (→) que define la causalidad. Si un evento *a* ocurre antes y causa un evento *b*, entonces el reloj lógico de *a* es estrictamente menor que el de *b* (L(a) < L(b)).

9. ¿Cómo funcionan los relojes lógicos de Lamport?
Cada proceso inicia un contador L=0. Ante un evento interno o envío de mensaje, hace L=L+1. El mensaje transporta ese valor L. Al recibir un mensaje con un valor T, el receptor actualiza su propio reloj haciendo L=max(L, T)+1.

10. ¿Qué significa que dos eventos sean concurrentes?
Significa que ocurren en procesos distintos sin que haya existido un intercambio de mensajes previo entre ellos que permita definir con certeza cuál causó al otro. Ninguno es causa directa del otro.

11. ¿Qué diferencias existen entre sincronizar relojes físicos y establecer un orden lógico de eventos?
La sincronización física ajusta el hardware/SO para que las máquinas compartan la misma hora del día. El orden lógico ignora la hora del día y estampa números secuenciales para asegurar la integridad y la relación causa-efecto a nivel de transacciones.

12. ¿Cómo funciona NTP y qué problemas permite resolver?
NTP es un protocolo jerárquico (estratos) que sincroniza relojes a través de internet. Resuelve los problemas de latencia asimétrica (haciendo múltiples muestreos estadísticos) y de tolerancia a fallos mediante redundancia de servidores.

2. Ejercicio 1: Algoritmo de Cristian

Ejecución realizada mediante contenedores Docker. Los datos corresponden a la primera muestra (Muestra 1/5), donde se observa el impacto del desfase inicial y la latencia asimétrica introducida por el arranque de la red del orquestador.

Cliente	Desfase inicial (s)	RTT (ms)	Corrección (ms)	Hora local corregida (estimada)
Client A	0.000	1671.50	84.81	2026-10-29T02:46:23.2570754+00:00
Client B	+8.000	3992.04	-6754.20	2026-10-29T02:46:24.4175682+00:00
Client C	-5.000	2975.70	5737.42	2026-10-29T02:46:23.9090161+00:00

Análisis: El RTT fue elevado en la primera medición debido al enrutamiento inicial de Docker. En las siguientes iteraciones (muestras 3, 4 y 5 registradas en consola), el RTT descendió a valores normales de red local (< 5 ms), logrando que la corrección tienda a 0.

3. Ejercicio 2: Algoritmo de Berkeley

El coordinador solicitó los offsets de tres nodos participantes. Los siguientes valores corresponden a los cálculos de la primera ronda de sincronización, logrando un consenso base para todos los contenedores.

Nodo	Offset inicial (ms)	Diferencia medida (ms)	Corrección enviada (ms)	Offset final resultante (ms)
Coordinador	0	0	-942.0	-942.0
Nodo 1	5000	5638.0	-6580.0	-1580.0
Nodo 2	10000	10351.2	-11293.2	-1293.2
Nodo 3	-20000	-19757.1	18815.1	-1184.9

Análisis: El promedio de las diferencias calculado por el coordinador fue de -942.0 ms. Tras enviar los ajustes, la dispersión inicial de los relojes (que abarcaba desde -20 segundos hasta +10 segundos) se redujo a una franja de consenso inferior a un segundo.

4. Ejercicio 3: Relojes Lógicos de Lamport

Flujo de ejecución documentado mediante peticiones HTTP REST entre tres microservicios (Orders, Inventory, Payments). Se evidencia el avance monotónico del contador L independientemente de la hora física subyacente.

Proceso (Servicio)	Evento	Hora Física (Log)	Reloj Lógico (Lamport)
OrdersService	pedido_recibido	2026-10-29T03:46:24.966Z	1
OrdersService	consulta_inventario_enviada	2026-10-29T03:46:25.039Z	2
InventoryService	mensaje_recibido (P001)	2026-10-29T03:46:26.034Z	3
InventoryService	inventario_verificado	2026-10-29T03:46:26.038Z	4
InventoryService	respuesta_enviada	2026-10-29T03:46:26.039Z	5
OrdersService	respuesta_inventario_recibida	2026-10-29T03:46:26.493Z	6
OrdersService	solicitud_pago_enviada	2026-10-29T03:46:26.499Z	7
PaymentsService	pago_recibido (ORD-001)	2026-10-29T03:46:27.899Z	8
PaymentsService	pago_procesado	2026-10-29T03:46:27.903Z	9
PaymentsService	respuesta_enviada	2026-10-29T03:46:27.904Z	10
OrdersService	respuesta_pago_recibida	2026-10-29T03:46:28.019Z	11
OrdersService	pedido_confirmado	2026-10-29T03:46:28.020Z	12

5. Ejercicio 4: Comparación de algoritmos

Característica	Cristian	Berkeley	Lamport
Tipo de reloj	Físico	Físico	Lógico (Enteros)
Objetivo principal	Sincronizar contra servidor externo	Consenso interno entre nodos	Orden causal de eventos
Requiere central	Sí (Servidor de Tiempo)	Sí (Coordinador)	No
Utiliza mensajes	Sí	Sí	Sí
Sincroniza hora física	Sí	Sí	No
Establece orden causal	No	No	Sí
Ante fallas	Falla si cae el servidor	Falla si cae el coordinador	Avanza independiente

Respuestas comparativas

1. ¿Qué algoritmo utilizaría para sincronizar los relojes de varios servidores?
NTP (evolución de Cristian) si se tiene acceso a internet y a relojes atómicos. Berkeley si el clúster estuviera completamente aislado y requiriera un consenso propio sin depender del exterior.

2. ¿Qué algoritmo utilizaría para establecer el orden causal de operaciones entre microservicios?
Relojes lógicos de Lamport o Relojes Vectoriales, inyectando el valor numérico en las cabeceras HTTP de las peticiones para trazar la travesía completa de la solicitud.

3. ¿Qué diferencias existen entre sincronización física y sincronización lógica?
La sincronización física lidia con milisegundos y latencia para que los *timestamps* del sistema operativo tengan coherencia temporal. La sincronización lógica garantiza de forma estricta la consistencia transaccional y la causalidad (qué evento disparó a otro), sin importar el hardware.

4. ¿Es posible combinar diferentes algoritmos en una misma arquitectura distribuida?
Sí, es la norma industrial. Se utiliza NTP para mantener sincronizados los *logs* físicos de los servidores, y relojes lógicos/vectoriales a nivel aplicativo para asegurar el orden causal en transacciones y bases de datos distribuidas.